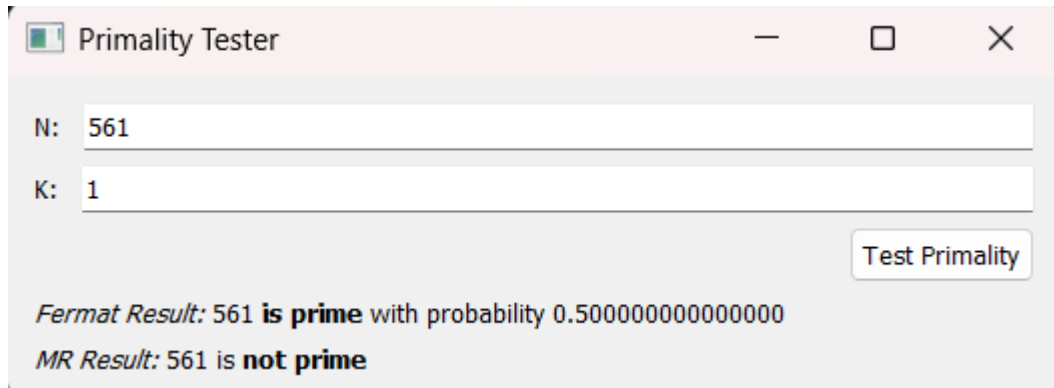


Bektur Akkabakov
CS 4412
Advanced Algorithms
Project 1: Primality Tester

To identify a case where the Fermat and Miller–Rabin algorithms disagree, I needed a composite integer that could potentially fool one of the algorithms. Carmichael numbers are ideal for this, because Fermat’s algorithm can mistakenly classify a Carmichael number as prime for up to half of the randomly chosen a_i values, where $1 \leq a_i \leq N$. The screenshot below shows such a disagreement.



As expected, the k-value must be very small to observe this outcome, since increasing the number of randomly chosen a_i values reduces the probability that Fermat fails to detect the composite nature of a Carmichael number.

Time and Space Complexity Analysis

1. Function: **modexp**

Time Complexity: $O(n^3)$

Space Complexity: $O(n)$

<i>if y==0:</i>	<i>O(1)</i>
<i>return 1</i>	
<i>z = mod_exp(x, y//2, N)</i>	<i>n recursive calls -> O(n) stack space</i>
<i>if y % 2 == 0:</i>	
<i>return z**2 % N</i>	<i>O(n²) time, O(1) space.</i>
<i>else:</i>	
<i>return x*z**2 % N</i>	<i>O(n²), O(1) space.</i>

Time complexity:

Each recursive call performs multiplication and modulo operation on n-bit integers, each costing $O(n^2)$. Thus, overall time complexity of the algorithm is:

$$T(n) = O(1) + n \cdot (n^2 + n^2) = O(1) + 2 \cdot n^3 = O(n^3)$$

Space complexity:

The function will execute n recursion calls. Each recursive call stores a constant number of n -bit variables, so the total auxiliary space is:

$$S(n) = n \cdot O(1) = O(n)$$

2. Function: **fermat**

Time Complexity: $O(k \cdot n^3)$

Space Complexity: $O(k \cdot n)$

```

r = []                                # stores k n-bit integers → O(k*n) space

for i in range(k):
    r.append(random.randint(1, N-1))  # k iterations → O(k) time

for a in r:                            # k iterations
    if mod_exp(a, N-1, N) != 1:        # O(n^3) time, O(n) stack space
        return 'composite'

return 'prime'

```

Time complexity:

The function performs k iterations, each invoking `mod_exp` once.

Each call to `mod_exp` runs in $O(n^3)$ time.

Thus, the overall time complexity is:

$$T(n, k) = O(k) + k \cdot O(n^3) = O(k \cdot n^3)$$

Space complexity:

The list `r` stores k n -bit integers, requiring $O(k \cdot n)$ space.

Each call to `mod_exp` uses $O(n)$ stack space, but calls are not nested, so this does not accumulate.

Therefore, the total auxiliary space complexity is:

$$S(n, k) = O(k \cdot n)$$

3. Function: **millar_rabin**

Time Complexity: $O(k \cdot n^4)$

Space Complexity: $O(k \cdot n)$

```

r = []                                # stores k n-bit integers → O(k*n) space

for i in range(k):
    r.append(random.randint(1, N-1))  # k iterations → O(k) time

for a in r:                            # k iterations
    if mod_exp(a, N-1, N) != 1:        # O(n^3) time, O(n) stack space
        return 'composite'

new_exp = N-1

while new_exp % 2 == 0:                # exponent halved → O(n) iterations

```

```

new_exp = new_exp // 2
res = mod_exp(a, new_exp, N)          # O(n^3) time, O(n) stack space
if res != 1:
    if res == N-1:
        break
    else:
        return 'composite'
return 'prime'

```

Time complexity:

The outer loop iterates k times. For each base, the initial Fermat check calls `mod_exp` once, costing $O(n^3)$

The inner loop repeatedly halves the exponent $N-1$, which executes at most $O(n)$ times. Each iteration invokes `mod_exp`, costing $O(n^3)$.

Thus, the total time complexity is:

$$T(n,k) = k * (n * n^3) = O(k * n^4)$$

Space complexity:

The list `r` stores k n -bit integers, requiring $O(k * n)$ space.

Each call to `mod_exp` uses $O(n)$ stack space, but calls are not nested, so this does not accumulate across iterations.

Therefore, the total auxiliary space complexity is:

$$S(n,k) = O(k * n)$$